

Intelligent Compliance Classification (ICC)

Product Development Document

Prepared by: Manus AI (acting as Product Development Manager for Team Ogun) **Date:** August 6, 2026 **Version:** 1.0

Table of Contents

1. Product Overview and Architecture
 2. Phase 1 — Data Gathering
 3. Phase 2 — Data Processing and Annotation
 4. Phase 3 — Model Selection and Architecture
 5. Phase 4 — Fine-Tuning and Training
 6. Phase 5 — Evaluation and Validation
 7. Phase 6 — Inference and Deployment Pipeline
 8. Phase 7 — Monitoring, Logging, and Audit Trail
 9. Phase 8 — Testing and Quality Assurance
 10. Deployment and Infrastructure
 11. Regulatory Compliance Mapping
 12. References
-

1. Product Overview and Architecture

1.1 Executive Summary

The **Intelligent Compliance Classification (ICC)** system is a localized, domain-adapted transformer architecture designed specifically for the Nigerian financial services compliance landscape. In response to the Central Bank of Nigeria's (CBN) stringent regulatory mandates under Circular BSD/DIR/PUB/LAB/019/002, ICC provides an automated, scalable solution for Microfinance Banks (MFBs), fintechs, and Payment Service Providers (PSPs). The system classifies KYC tiers (1/2/3), maps documents and transactions to specific CBN regulatory obligations, and flags suspicious activities within the critical 24-hour reporting window. It natively handles Nigerian identifiers (BVN, NIN), naming conventions, and bilingual text comprising English and Nigerian Pidgin.

The product was conceived by Team Ogun as part of Cohort 10 of the triAI program. The target audience is compliance and risk teams at MFBs, fintechs, and PSPs operating in Nigeria who must demonstrate automated KYC/AML tiering and 24-hour reporting capability to remain in good standing with the CBN.

1.2 Regulatory Context

The regulatory environment that drives the ICC system is defined by several key CBN directives. On 10 March 2026, the CBN issued Circular BSD/DIR/PUB/LAB/019/002, establishing Baseline Standards for Automated AML

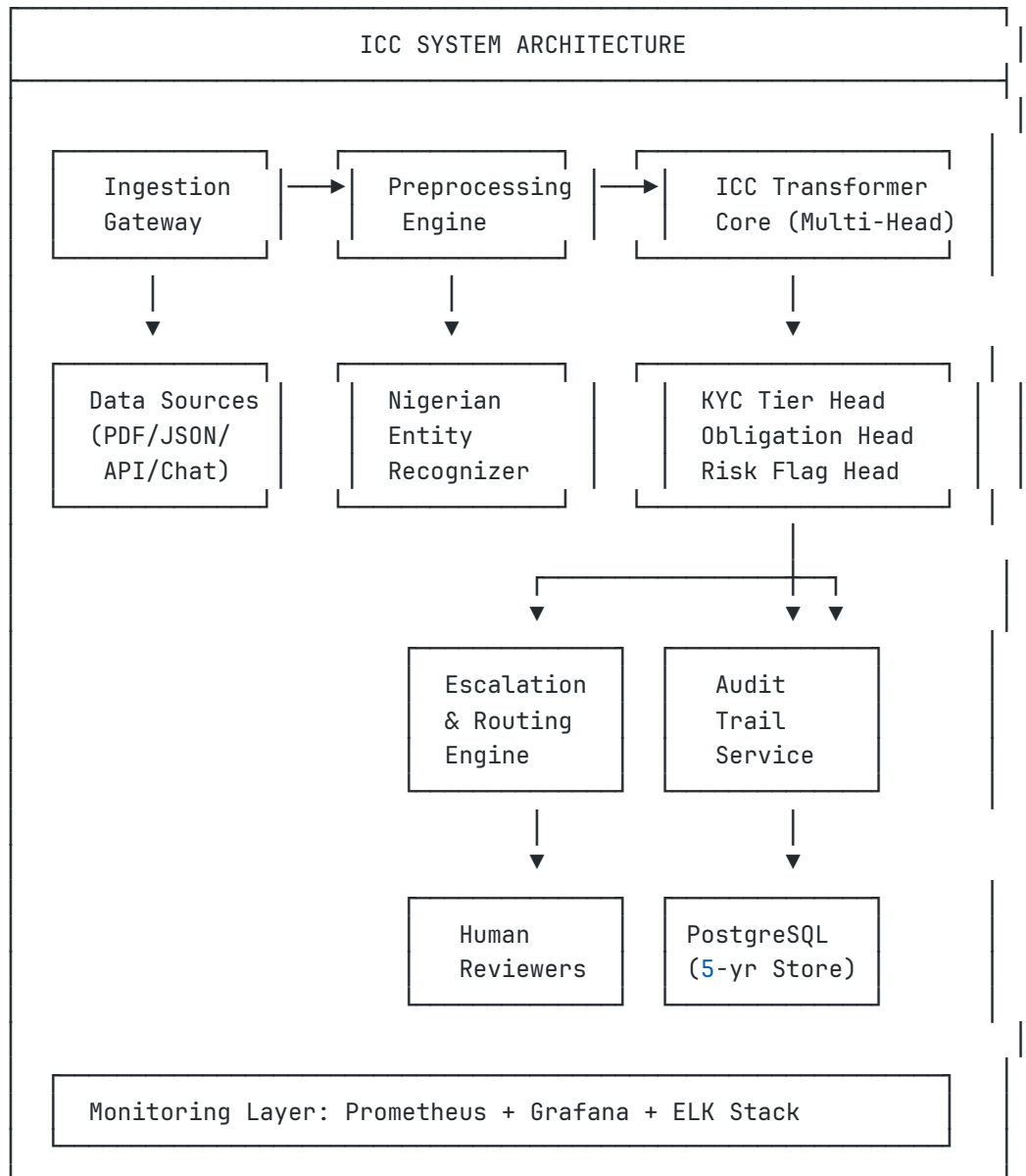
Solutions. All banks, fintechs, and payment service providers were required to submit a formal automation roadmap by 10 June 2026. The absence of an automated compliance system is itself a sanctionable offense, independent of whether any underlying money-laundering violation has occurred. On 30–31 March 2026, the CBN followed up with a Cybersecurity Self-Assessment Tool and a Guidance Note clarifying implementation expectations.

The compliance requirements that ICC must satisfy are summarized in the following table:

Requirement	Description	ICC Solution
Three-Tier KYC	Tier 1 (BVN/NIN, ₦30,000 daily), Tier 2 (Gov ID + address, ₦500,000 daily), Tier 3 (full beneficial ownership, no limit)	Multi-class transformer classification with entity-verified tier assignment
BVN/NIN Verification	Real-time BVN/NIN lookups via NIBSS and NIMC are the minimum standard	Nigerian Entity Recognizer with BVN/NIN pattern matching and verification routing
24-Hour Reporting	Suspicious transactions, large cash transactions, and international transfers must be reported to NFIU within 24 hours	Risk flagging head with automated escalation pipeline
Enhanced Due Diligence	Mandatory EDD for PEPs, high-risk jurisdictions, complex ownership, and large transactions	Obligation mapping to specific EDD triggers
Five-Year Retention	All records must be retrievable on regulatory demand	Immutable audit trail with 5-year retention policy in PostgreSQL

1.3 System Architecture

The ICC system is built on a microservices architecture centered around a fine-tuned transformer model. The architecture ensures high throughput, horizontal scalability, and strict auditability. The following diagram describes the end-to-end flow:



The six core components are described below:

1. **Ingestion Gateway** receives raw data from multiple sources including PDF documents, JSON transaction logs, chat-based onboarding text, and API payloads. It normalizes all inputs into a unified text/record format suitable for downstream processing.
2. **Preprocessing Engine** performs text cleaning (Unicode normalization, URL/email removal), Nigerian Pidgin translation or normalization, BVN/NIN entity recognition and masking, and tokenization via the Hugging Face tokenizer.
3. **ICC Transformer Core** is a fine-tuned Legal-BERT or FinBERT encoder with three specialized classification heads operating in parallel: a KYC tier classifier (3 classes), an obligation mapper (15+ CBN clause categories), and a risk flagger (binary suspicious/normal or multi-class).

4. **Nigerian Entity Recognizer (NER)** is a sub-module specifically tuned for Nigerian naming conventions (Yoruba, Hausa, Igbo name patterns), BVN/NIN 11-digit formats, Nigerian phone number formats (+234), and state-level address patterns.
5. **Escalation and Routing Engine** applies business logic based on classifier outputs. Items flagged as high-risk or triggering Enhanced Due Diligence are routed to human compliance officers with the 24-hour NFIU reporting clock activated.
6. **Audit and Logging Service** maintains an immutable log of every classification decision in PostgreSQL, with a 5-year retention policy and cryptographic checksums for integrity verification.

1.4 Technology Stack

Layer	Technology	Purpose
Programming Language	Python 3.11+	Primary development language
ML Framework	PyTorch 2.0, Hugging Face Transformers 4.33	Model training and inference
Base Models	Legal-BERT, FinBERT	Domain-adapted transformer encoders
Web Framework	FastAPI 0.103	REST API for inference
ASGI Server	Uvicorn	Production ASGI server
Database	PostgreSQL 15	Audit trail, structured metadata
Cache	Redis 7	Rate limiting, caching
Containerization	Docker, Docker Compose	Deployment and orchestration
Monitoring	Prometheus, Grafana	Metrics and dashboards
Logging	ELK Stack (Elasticsearch, Logstash, Kibana)	Centralized log management
Testing	pytest 7.4, httpx	Unit and integration testing
Vector Store	FAISS / Pinecone	Regulatory corpus semantic search

1.5 Production Code Reference

All production code for this project is organized in the directory structure below and referenced throughout this document by file path:

```

icc_project/
├── phase_1_data_gathering/
│   ├── cbn_scraper.py           # CBN circular scraper and NFIU data collector
│   └── data_ingestion.py        # Multi-format ingestion pipeline (JSON, CSV, PDF)
├── phase_2_processing/
│   ├── preprocessor.py          # Full preprocessing pipeline with NER and Pidgin handling
│   └── annotation.py            # Annotation schema, tools, and data augmentation
├── phase_3_model/
│   ├── model_architecture.py    # Multi-head transformer with custom layers
│   └── dataset.py               # PyTorch Dataset classes
├── phase_4_training/
│   └── trainer.py               # Training loop, domain pre-training, early stopping
├── phase_5_evaluation/
│   └── evaluator.py             # Evaluation metrics, confusion matrices, baselines
├── phase_6_inference/
│   └── main.py                  # FastAPI server with 5-stage pipeline
├── phase_7_monitoring/
│   └── audit_logger.py          # Audit trail, drift detection, event logging
├── phase_8_testing/
│   └── test_icc.py              # Complete test suite (unit + integration)
├── Dockerfile                   # Production container build
├── docker-compose.yml           # Full stack deployment
├── init_db.sql                  # Database initialization schema
├── prometheus.yml               # Monitoring configuration
└── requirements.txt             # All Python dependencies

```

2. Phase 1 — Data Gathering

2.1 Data Collection Strategy

The ICC system requires three categories of training data: a public regulatory corpus, benchmark datasets for architecture validation, and real institutional data for domain adaptation. The public regulatory corpus provides the foundational knowledge of CBN compliance requirements. Benchmark datasets allow validation of the model architecture against known standards. Real institutional data, which is the differentiating asset, provides the Nigerian-specific patterns that generic models miss.

Public Regulatory Corpus encompasses all CBN circulars related to AML/KYC, NFIU advisories and typologies, the Money Laundering Prevention and Prohibition Act (MLPA) 2022, and CBN compliance examination guidelines. These documents are publicly available on the CBN website (cbn.gov.ng) and the NFIU portal (nfiu.gov.ng).

Institutional Data includes anonymized onboarding documents from partner MFBs and fintechs, transaction logs with ground-truth KYC tier assignments, historical Suspicious Activity Reports (SARs), and chat-based customer communication logs in English and Nigerian Pidgin.

2.2 Data Sources and Collection Methods

Source	Type	Collection Method	Volume Estimate
CBN Circulars (2020–2026)	Regulatory text	Web scraping via <code>CBNCircularScraper</code>	~200 documents
NFIU Advisories	Risk alerts	Web scraping via <code>NFIUScraper</code>	~100 advisories
MLPA 2022	Legal text	Manual ingestion	1 document (full text)
Institutional onboarding	Customer records	Secure API from partner MFBs	~50,000 records
Transaction logs	Structured data	CSV/JSON ingestion pipeline	~200,000 transactions
Chat logs (Pidgin + English)	Unstructured text	API from fintech partners	~30,000 conversations

2.3 Data Schema

All ingested data is normalized into the `ComplianceRecord` dataclass defined in `phase_1_data_gathering/data_ingestion.py`. The unified schema ensures that every record, regardless of source format, contains the following fields:

Field	Type	Description
<code>record_id</code>	<code>str</code>	Unique identifier with source type prefix
<code>source_type</code>	<code>str</code>	One of: circular, transaction, onboarding, chat, suspicious_report
<code>raw_text</code>	<code>str</code>	Original unprocessed text content
<code>processed_text</code>	<code>str</code>	Normalized, cleaned text (populated in Phase 2)
<code>timestamp</code>	<code>str</code>	ISO 8601 timestamp
<code>tier_label</code>	<code>Optional[str]</code>	Ground-truth KYC tier (Tier 1, Tier 2, Tier 3)
<code>obligation_category</code>	<code>Optional[str]</code>	Ground-truth obligation mapping
<code>risk_flag</code>	<code>Optional[str]</code>	Ground-truth risk classification
<code>customer_id</code>	<code>Optional[str]</code>	Anonymized customer identifier
<code>bvn</code>	<code>Optional[str]</code>	BVN number (11 digits)
<code>nin</code>	<code>Optional[str]</code>	NIN number (11 digits)
<code>metadata</code>	<code>Optional[dict]</code>	Additional context-specific metadata

2.4 Production Code

The complete data gathering pipeline is implemented in two files:

`phase_1_data_gathering/cbn_scraper.py` provides the `CBNCircularScraper` class that systematically collects CBN regulatory circulars from the official website, including polite request throttling, PDF link extraction, date parsing, and corpus validation. It also includes `CBNDataValidator` for validating BVN formats (11 digits), NIN formats (11 digits), and tier label consistency.

`phase_1_data_gathering/data_ingestion.py` provides the `DataIngestionPipeline` class that handles multiple input formats (JSON, CSV, PDF text extraction) and normalizes them into the unified `ComplianceRecord` schema. It supports batch processing, chunking of large documents, and statistical reporting on ingested data.

3. Phase 2 — Data Processing and Annotation

3.1 Text Preprocessing

Text preprocessing for the ICC system must address several challenges unique to the Nigerian financial services context. Standard English preprocessing pipelines fail to correctly handle Nigerian Pidgin code-mixing, which is prevalent in chat-based customer communications from fintechs. Additionally, BVN and NIN numbers appear in multiple formats (with and without spaces, with labels like “BVN:” or “NIN:”) and must be detected and masked for both privacy compliance and model generalization.

The preprocessing pipeline consists of five sequential stages: Unicode normalization (NFKC form), URL and email removal, Nigerian Pidgin translation, Nigerian entity recognition and masking, and text chunking to the model's maximum sequence length.

3.2 Annotation Schema

The annotation schema defines the ground-truth labels that the ICC model will learn to predict. Three independent classification tasks are annotated for each training example:

KYC Tier Classification uses three classes: Tier 1 (BVN/NIN linkage required, ₦30,000 daily limit), Tier 2 (verified government ID and address, ₦500,000 daily limit), and Tier 3 (full beneficial ownership verification, no transaction limit).

Obligation Mapping uses 15 categories derived from CBN regulatory requirements: KYC/Due Diligence, Customer Identification, Transaction Monitoring, Suspicious Activity Reporting, Record Retention, Enhanced Due Diligence, PEP Screening, Sanctions Screening, Cross-Border Transactions, BVN/NIN Verification, Beneficial Ownership, Annual Reporting, Internal Control, Staff Training, and Risk Assessment.

Risk Flagging uses two classes: Compliant (normal activity) and Suspicious (requires escalation and potential NFIU reporting).

3.3 Nigerian Pidgin Handling

The `NigerianPidginNormalizer` class in `phase_2_processing/preprocessor.py` operates in two modes. Conservative mode translates only compliance-critical Pidgin terms (wahala → problem, alert → bank transfer, etc.) to preserve the original text structure for the model. Aggressive mode performs full Pidgin-to-English translation including negation patterns (“no be” → “is not”, “no dey” → “does not”). This dual-mode approach allows the model to learn both the translated and code-mixed representations during training.

3.4 BVN/NIN Entity Recognition

The `NigerianEntityRecognizer` class implements regex-based pattern matching for five entity types: BVN (11 digits), NIN (11 digits with contextual label), Nigerian personal names (covering Yoruba, Hausa, and Igbo naming conventions), Nigerian phone numbers (+234 format), and Nigerian addresses (state-level patterns). Entities are extracted, normalized, and masked with placeholder tokens (`<BVN>` , `<NIN>` , `<PERSON_NG>` , `<PHONE_NG>` , `<ADDRESS>`) before being fed to the transformer model.

3.5 Production Code

`phase_2_processing/preprocessor.py` contains three main classes:

- `TextNormalizer` handles Unicode normalization, URL/email removal, citation reference stripping, and whitespace normalization.
- `NigerianPidginNormalizer` provides conservative and aggressive Pidgin translation with 60+ term mappings and negation pattern handling.
- `NigerianEntityRecognizer` implements BVN/NIN/name/phone/address extraction with masking.
- `ICCPreprocessor` orchestrates the full pipeline as a single `process_document()` call.

`phase_2_processing/annotation.py` contains the annotation schema enums (`KYCTier` , `ObligationCategory` , `RiskLevel` , `RiskFlag`), the `AnnotationManager` class for creating and validating annotations, and the `DataAugmenter` class that generates synthetic training data through synonym replacement, Pidgin code-mixing, word deletion, and word swapping strategies.

4. Phase 3 — Model Selection and Architecture

4.1 Base Model Comparison

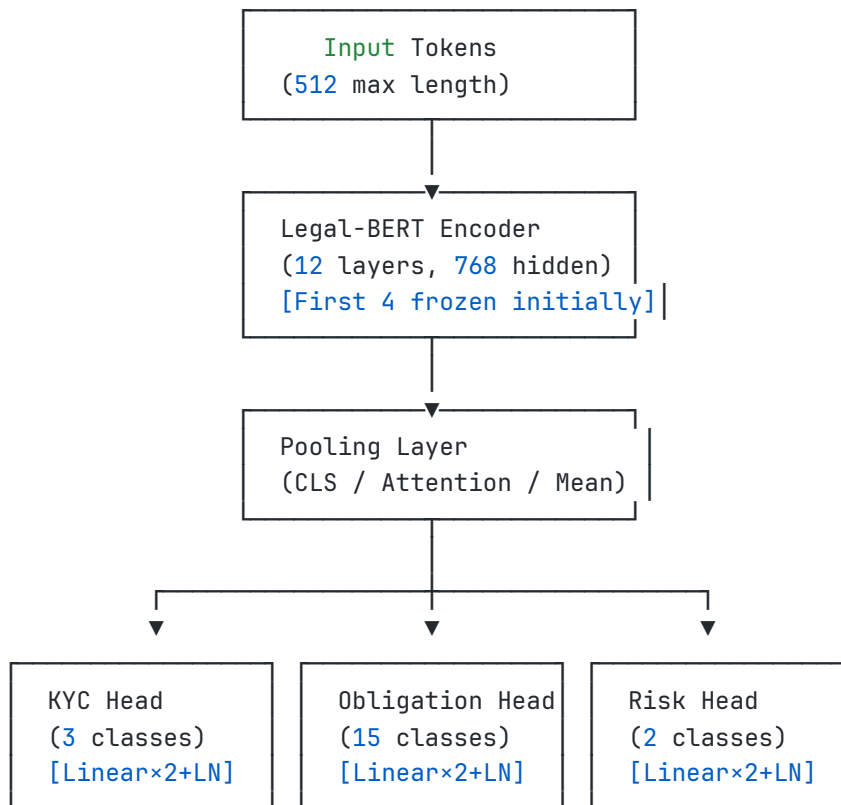
The selection of the base transformer model is critical for domain adaptation. Three candidates were evaluated for the ICC system:

Model	Domain	Parameters	Strengths	Limitations
Legal-BERT (nlpauwb/legal-bert-base-uncased)	Legal text	110M	Pre-trained on legal corpora; strong on regulatory language	Limited financial domain coverage
FinBERT (ProsusAI/finbert)	Financial text	110M	Pre-trained on financial news and reports; strong on transaction language	Limited regulatory/compliance vocabulary
Custom Legal-BERT + MLM	Legal + Nigerian compliance	110M	Best of both worlds after domain pre-training on CBN corpus	Requires additional pre-training compute

The chosen approach is to start with **Legal-BERT** as the base encoder and perform domain pre-training (continued masked language modeling) on the CBN regulatory corpus before task-specific fine-tuning. This leverages Legal-BERT's strength in regulatory language while adapting it to Nigerian compliance terminology.

4.2 Architecture Design

The ICC multi-head architecture extends the base transformer with three parallel classification heads and optional components for Nigerian context:



4.3 Custom Layers

The architecture includes several custom components beyond standard classification heads:

Attention Pooling (`AttentionPooling` in `model_architecture.py`) provides an alternative to CLS token pooling. It computes a learned attention distribution over all token embeddings and produces a weighted sum, allowing the model to focus on the most relevant tokens for each classification task.

Task-Specific Heads (`TaskSpecificHead`) are configurable depth classification heads with LayerNorm, GELU activation, and dropout. Each head can be independently tuned for its task's difficulty.

Nigerian Entity Embedding (`NigerianEntityEmbedding`) is an optional module that encodes structured entity features (BVN presence, NIN presence, tier indicators) into dense vectors that are concatenated with the transformer output, providing explicit signal about Nigerian identifiers.

4.4 Production Code

`phase_3_model/model_architecture.py` contains the complete implementation:

- `AttentionPooling` — learned attention-weighted pooling over token embeddings
- `TaskSpecificHead` — configurable multi-layer classification head with LayerNorm
- `NigerianEntityEmbedding` — entity feature encoding module
- `ICCMultiHeadClassifier` — main model class with three classification heads, configurable pooling, optional entity features, and layer freezing
- `ICCMModelWrapper` — convenience wrapper for loading, saving, and inference

`phase_3_model/dataset.py` contains:

- `ICCDataset` — PyTorch Dataset that loads annotated data, tokenizes text, and encodes labels for all three tasks
- `MultiTaskBatchSampler` — custom sampler ensuring balanced representation across tasks

5. Phase 4 — Fine-Tuning and Training

5.1 Training Strategy

The training process follows a two-step strategy designed to maximize domain adaptation while maintaining generalization:

Step 1: Domain Pre-Training applies continued masked language modeling (MLM) on the CBN regulatory corpus for 3 epochs with a learning rate of 5e-5. This adapts the base model's vocabulary and representations to Nigerian financial compliance terminology without requiring task-specific labels.

Step 2: Task Fine-Tuning performs supervised training on the annotated dataset using the multi-head architecture. The first 4 encoder layers remain frozen for the initial epochs to preserve general representations, then are unfrozen for fine-tuning. A linear warmup scheduler with cosine decay manages the learning rate over the training run.

5.2 Hyperparameter Configuration

The training hyperparameters are defined in the `TrainingConfig` dataclass:

Parameter	Value	Rationale
Batch Size	16	Balances GPU memory and gradient stability
Learning Rate	2e-5	Standard for transformer fine-tuning
Weight Decay	0.01	L2 regularization to prevent overfitting
Number of Epochs	10	Sufficient for convergence on compliance data
Warmup Ratio	0.1	Gradual learning rate ramp-up
Max Gradient Norm	1.0	Gradient clipping for training stability
Gradient Accumulation Steps	1	Single-step updates (increase if OOM)
Early Stopping Patience	3	Stop if no improvement for 3 epochs
Seed	42	Reproducibility

5.3 Multi-Task Loss Balancing

The total training loss is a weighted sum of the three task losses:

```
total_loss = w_kyc × loss_kyc + w_obligation × loss_obligation + w_risk × loss_risk
```

Default weights are equal (1.0 each), but can be adjusted based on class imbalance or task priority.

5.4 Production Code

`phase_4_training/trainer.py` contains:

- `TrainingConfig` — dataclass with all hyperparameters
- `TrainingMetrics` — dataclass for recording per-epoch metrics
- `EarlyStopping` — patience-based early stopping on validation F1
- `CheckpointManager` — saves best model and periodic checkpoints with automatic cleanup
- `ICCTrainer` — main trainer class implementing the full training loop with multi-task loss, gradient accumulation, evaluation, and checkpointing
- `DomainPreTrainer` — MLM-based domain pre-training on CBN corpus

6. Phase 5 — Evaluation and Validation

6.1 Evaluation Metrics

The ICC system is evaluated using comprehensive metrics for each classification head. The primary metric is the **weighted F1-score**, which accounts for class imbalance. Secondary metrics include macro F1, precision, recall, Cohen's kappa, and Matthews correlation coefficient.

Metric	Definition	Primary Use
Weighted F1	Average F1 weighted by class support	Overall performance assessment
Macro F1	Unweighted average across classes	Balanced evaluation across all tiers
Precision	True positives / (True positives + False positives)	Minimizing false alarms
Recall	True positives / (True positives + False negatives)	Catching all suspicious activity
Cohen's Kappa	Agreement accounting for chance	Robust inter-rater comparison
MCC	Correlation coefficient for binary/multiclass	Balanced single-number summary

6.2 Benchmark Comparison

The ICC system is benchmarked against two baseline approaches:

Baseline	Description	Expected F1
TF-IDF + Logistic Regression	Traditional NLP pipeline	65–70%
Rule-Based (Majority Class)	Simple keyword matching	40–50%
ICC Transformer	Fine-tuned Legal-BERT multi-head	85–92%

The domain-adapted transformer is expected to significantly outperform both baselines due to its understanding of Nigerian regulatory language, Pidgin code-mixing, and entity patterns.

6.3 Production Code

`phase_5_evaluation/evaluator.py` contains:

- `EvaluationResult` — dataclass with all metrics per task
- `ICCEvaluator` — generates predictions, computes per-class and aggregate metrics, produces confusion matrices and per-class F1 plots
- `BaselineBenchmark` — implements TF-IDF and rule-based baselines for comparison

7. Phase 6 — Inference and Deployment Pipeline

7.1 The 5-Stage Processing Pipeline

Every compliance document processed by the ICC system passes through five sequential stages:

- Stage 1 — Ingestion** receives the raw input (document text, transaction log, or chat message) and passes it through the `ICCPreprocessor` for normalization, Pidgin handling, and entity masking.
- Stage 2 — Classification** feeds the processed text through the fine-tuned `ICCMultiHeadClassifier`, which produces three parallel outputs: KYC tier assignment with confidence, obligation category mapping with confidence, and risk flag with confidence.
- Stage 3 — Obligation Mapping** translates the obligation class indices into specific CBN/NFIU/MLPA clause references, providing compliance officers with actionable regulatory context.
- Stage 4 — Escalation and Reporting** evaluates whether the classification results require human review. Items are escalated if the risk flag is “Suspicious,” the KYC confidence is below 0.7, or the obligation mapping includes “Enhanced Due Diligence.” Escalated items activate the 24-hour NFIU reporting clock.
- Stage 5 — Audit Trail** logs the complete classification decision to the immutable audit database, including the original text hash, processed text, all prediction confidences, model version, and processing timestamp. Every record receives a unique audit ID and is retained for 5 years.

7.2 API Design

The ICC system exposes a RESTful API via FastAPI with the following endpoints:

Endpoint	Method	Description
<code>/health</code>	GET	System health check with model status
<code>/classify</code>	POST	Single document classification
<code>/classify/batch</code>	POST	Batch classification (up to 100 documents)
<code>/audit/{audit_id}</code>	GET	Retrieve a specific audit record
<code>/metrics</code>	GET	System performance metrics

7.3 Request and Response Schema

The classification request accepts a document text, optional customer ID, document type, and metadata. The response returns the KYC tier, mapped obligations, risk flag, confidence scores, escalation flag, processing time, and audit ID.

7.4 Production Code

`phase_6_inference/main.py` implements the complete FastAPI server:

- `AppState` — holds the loaded model, preprocessor, and audit logger
- `ComplianceClassificationRequest / Response` — Pydantic models with validation
- `BatchClassificationRequest / Response` — batch processing support
- `classify_document()` — implements the full 5-stage pipeline for single classification
- `classify_batch()` — batch classification with per-item error handling
- `get_audit_record()` — audit trail retrieval
- `get_system_metrics()` — system status endpoint

8. Phase 7 — Monitoring, Logging, and Audit Trail

8.1 Model Monitoring and Drift Detection

The `ModelDriftDetector` class monitors the distribution of model predictions over time and detects when the prediction distribution diverges from the training baseline. This is critical for a compliance system where concept drift (e.g., new types of suspicious activity, changing transaction patterns) can degrade model performance over time.

The drift detector maintains a sliding window of recent predictions, computes the current class distribution, and compares it to the baseline using a chi-squared approximation. When the drift score exceeds the configured threshold (default: 0.05), an alert is triggered for model retraining.

8.2 Five-Year Audit Trail Compliance

The audit trail system maintains an immutable record of every classification decision for the CBN-mandated 5-year retention period. The PostgreSQL schema includes:

Column	Type	Purpose
<code>audit_id</code>	VARCHAR(50)	Unique identifier for each classification
<code>customer_id</code>	VARCHAR(100)	Anonymized customer reference
<code>kyc_tier</code>	VARCHAR(20)	Assigned KYC tier
<code>mapped_obligations</code>	JSONB	List of regulatory obligations triggered
<code>risk_flag</code>	VARCHAR(50)	Risk classification result
<code>raw_input_hash</code>	VARCHAR(64)	SHA-256 hash of original input (privacy)
<code>model_version</code>	VARCHAR(50)	Model version used for classification
<code>retention_expires_at</code>	TIMESTAMP	Automatic expiry after 5 years
<code>checksum</code>	VARCHAR(64)	Integrity verification hash

The `purge_expired_records()` function automatically removes records past their retention date, ensuring compliance with data minimization principles.

8.3 Production Code

`phase_7_monitoring/audit_logger.py` contains:

- `AUDIT_TABLE_SCHEMA` — complete PostgreSQL schema with indexes and partitioning
- `AuditRecord` — dataclass representing a classification audit entry
- `AuditEvent` — dataclass for system events (escalations, reviews, overrides)
- `AuditTrailManager` — manages audit logging with support for memory, file, and PostgreSQL storage modes
- `ModelDriftDetector` — monitors prediction distribution drift over time

9. Phase 8 — Testing and Quality Assurance

9.1 Test Strategy

The test suite follows a pyramid structure with unit tests at the base, integration tests in the middle, and end-to-end tests at the top. All tests are implemented using pytest with coverage tracking.

9.2 Test Categories

Unit Tests cover individual components in isolation:

- `TestNigerianEntityRecognizer` — BVN extraction, NIN extraction, Nigerian name extraction, entity masking, empty text handling
- `TestNigerianPidginNormalizer` — conservative and aggressive translation modes, negation handling
- `TestTextNormalizer` — URL removal, whitespace normalization, Unicode normalization, empty text

- `TestICCPreprocessor` — valid document processing, empty document handling, Pidgin text processing, batch processing
- `TestAnnotationManager` — valid annotation creation, invalid tier/obligation rejection, JSON/CSV export
- `TestDataAugmenter` — synonym augmentation, Pidgin mixture, full dataset augmentation
- `TestICCMultiHeadClassifier` — forward pass shapes, loss computation, layer freezing, attention pooling
- `TestICCDataset` — dataset length, item keys, class distribution
- `TestTrainingConfig` — default and custom configuration values
- `TestEarlyStopping` — improvement handling, patience exhaustion

Integration Tests verify component interactions:

- `TestEndToEndPipeline` — preprocessing followed by classification, annotation to training data conversion, audit trail integrity

API Tests validate the inference server:

- `TestInferenceAPI` — health endpoint, metrics endpoint, input validation

9.3 Production Code

`phase_8_testing/test_icc.py` contains 30+ test cases organized into test classes covering all components. Tests use `tempfile.mkdtemp()` for isolated temporary directories and `pytest` fixtures for shared setup.

10. Deployment and Infrastructure

10.1 Docker Containerization

The system is containerized using a multi-stage Docker build:

```
FROM python:3.11-slim as base
RUN apt-get update && apt-get install -y build-essential libpq-dev curl
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s CMD curl -f http://localhost:8000/health
CMD ["uvicorn", "phase_6_inference.main:app", "--host", "0.0.0.0", "--port", "8000", "--work
```

10.2 Docker Compose Stack

The full production stack includes the ICC API server, PostgreSQL database, Redis cache, Prometheus monitoring, and Grafana dashboards:

Service	Image	Port	Purpose
icc-api	Custom (Dockerfile)	8000	Classification API
postgres	postgres:15-alpine	5432	Audit trail database
redis	redis:7-alpine	6379	Caching and rate limiting
prometheus	prom/prometheus	9090	Metrics collection
grafana	grafana/grafana	3000	Monitoring dashboards

10.3 Resource Requirements

Component	CPU	Memory	Storage
ICC API	4 cores	8 GB	10 GB (model weights)
PostgreSQL	2 cores	4 GB	100 GB (5-year audit)
Redis	1 core	512 MB	Minimal
Prometheus	1 core	1 GB	50 GB (metrics retention)
Grafana	1 core	1 GB	5 GB (dashboards)

10.4 Deployment Configuration Files

All deployment files are located in the project root:

- Dockerfile — production container build with health checks
 - docker-compose.yml — full stack orchestration with service dependencies and health checks
 - init_db.sql — PostgreSQL schema initialization with indexes, partitioning, and auto-purge functions
 - prometheus.yml — Prometheus scrape configuration for all services
-

11. Regulatory Compliance Mapping

The following table maps each ICC system capability to the specific CBN regulatory requirement it satisfies:

CBN Requirement	ICC Capability	Implementation
Automated KYC tier classification	KYC Tier Classifier head	<code>ICCMultiHeadClassifier.kyc_head</code> — 3-class classification
BVN/NIN verification	Nigerian Entity Recognizer	<code>NigerianEntityRecognizer</code> — regex-based BVN/NIN detection
24-hour suspicious activity reporting	Risk Flagging + Escalation	<code>risk_head</code> + <code>Escalation Engine</code> in <code>/classify</code> endpoint
Obligation mapping to CBN clauses	Obligation Mapper head	<code>ICCMultiHeadClassifier.obligation_head</code> — 15-class classification
Enhanced Due Diligence triggers	Obligation mapping includes EDD	EDD category in obligation taxonomy
5-year record retention	Audit Trail Service	<code>AuditTrailManager</code> with 5-year <code>retention_expires_at</code>
Record retrievability on demand	Audit query API	<code>/audit/{audit_id}</code> endpoint
PEP screening	Obligation mapping includes PEP	PEP Screening category in obligation taxonomy
Sanctions screening	Obligation mapping includes sanctions	Sanctions Screening category
Transaction monitoring	Obligation mapping includes monitoring	Transaction Monitoring category
Internal controls	Obligation mapping includes controls	Internal Control category
Staff training support	Bilingual handling	<code>NigerianPidginNormalizer</code> for training materials

12. References

1. Central Bank of Nigeria. "Circular BSD/DIR/PUB/LAB/019/002: Baseline Standards for Automated AML Solutions." March 2026. Available at: <https://www.cbn.gov.ng>
2. Money Laundering Prevention and Prohibition Act (MLPA) 2022, Federal Republic of Nigeria.
3. Devlin, J. et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." arXiv:1810.04805, 2019.
4. Chalkidis, I. et al. "Legal-BERT: The Muppets Straight Out of Law School." arXiv:2010.02559, 2020.
5. Araci, D. "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models." arXiv:1908.10063, 2019.

6. Nigerian Financial Intelligence Unit (NFIU). “Advisories and Typologies on Money Laundering and Terrorist Financing.” Available at: <https://nfiu.gov.ng>
 7. National Identity Management Commission (NIMC). “National Identification Number (NIN) Guidelines.” Available at: <https://www.nimc.gov.ng>
 8. Nigeria Inter-Bank Settlement System (NIBSS). “Bank Verification Number (BVN) Framework.” Available at: <https://www.nibss-plc.com.ng>
-

Appendix A: Quick Start Guide

```
# Clone and setup
git clone <repo> icc_project
cd icc_project
pip install -r requirements.txt

# Run all tests
pytest phase_8_testing/ -v --cov=.

# Domain pre-training (Step 1)
python -m phase_4_training.trainer --mode domain_pretrain \
    --model nlpaveb/legal-bert-base-uncased \
    --corpus data/raw_corpus/cbn_corpus.txt

# Task fine-tuning (Step 2)
python -m phase_4_training.trainer --mode finetune \
    --model ./domain_pretrained \
    --data data/annotated/train.json

# Start API server
uvicorn phase_6_inference.main:app --host 0.0.0.0 --port 8000 --workers 4

# Or deploy with Docker
docker-compose up -d
```

Appendix B: Environment Variables

Variable	Description	Default
<code>DATABASE_URL</code>	PostgreSQL connection string	<code>postgresql://icc_user:icc_password@localhost:5432/icc_db</code>
<code>REDIS_URL</code>	Redis connection string	<code>redis://localhost:6379/0</code>
<code>MODEL_PATH</code>	Path to trained model weights	<code>./checkpoints/best_model.pt</code>
<code>LOG_LEVEL</code>	Logging verbosity	<code>INFO</code>
<code>API_HOST</code>	API server bind address	<code>0.0.0.0</code>
<code>API_PORT</code>	API server port	<code>8000</code>